

QUESTION 1

CONFIRMED HOTEL RESERVATIONS IN BEACH DESTINATIONS

Problem Statement

StayEase Hotels is preparing a special summer marketing campaign for its coastal properties. The marketing team wants to identify customers who have **confirmed reservations in beach destinations** so they can send exclusive vacation packages, discount coupons, and loyalty rewards.

Write an SQL query to retrieve the details of customers who have confirmed reservations in beach destination hotels.

The report should display:

- customer_name – Name of the customer
- hotel_name – Name of the hotel
- city – City where the hotel is located

The result must be sorted alphabetically by customer name.

Table Schema

Table: customers

Column Name	Data Type	Description
customer_id	INT	Unique customer identifier
customer_name	VARCHAR(100)	Customer name
email	VARCHAR(100)	Customer email
city	VARCHAR(50)	Customer city

Table: hotels

Column Name	Data Type	Description
hotel_id	INT	Unique hotel identifier
hotel_name	VARCHAR(100)	Hotel name
city	VARCHAR(50)	Hotel city
hotel_type	VARCHAR(20)	Beach, City, Mountain

Table: reservations

Column Name	Data Type	Description
reservation_id	INT	Unique reservation identifier
customer_id	INT	References customers(customer_id)
hotel_id	INT	References hotels(hotel_id)
reservation_status	VARCHAR(20)	Confirmed, Cancelled
reservation_date	DATE	Reservation date

Sample Input

Sample rows from customers

customer_id	customer_name
1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan
5	Emma Patel

Sample rows from hotels

hotel_id	hotel_name	city	hotel_type
1	Ocean View Resort	Goa	Beach
2	Royal Stay Hotel	Delhi	City
3	Palm Paradise Resort	Maldives	Beach
4	Hill Retreat	Manali	Mountain

Sample rows from reservations

reservation_id	customer_id	hotel_id	reservation_status
1	1	1	Confirmed
2	2	2	Confirmed
3	3	3	Confirmed
4	4	4	Confirmed
5	5	1	Cancelled

Expected Output Format

customer_name hotel_name city

Sample Output

customer_name	hotel_name	city
Alice Sharma	Ocean View Resort	Goa
Charlie Roy	Palm Paradise Resort	Maldives

Explanation

Reservation 1

Confirmed Reservation ✓

Beach Destination ✓

Included

Reservation 2

Confirmed Reservation ✓

Beach Destination ✗

Excluded

(City hotel)

Reservation 3

Confirmed Reservation ✓

Beach Destination ✓

Included

Reservation 4

Confirmed Reservation ✓

Beach Destination ✗

Excluded

(Mountain hotel)

Reservation 5

Cancelled Reservation X

Excluded

Approach

Step 1: Join the Tables

We need data from:

customers → reservations → hotels

Join Conditions:

c.customer_id = r.customer_id

r.hotel_id = h.hotel_id

Step 2: Filter Confirmed Reservations

Only reservations with:

reservation_status = 'Confirmed'

should be included.

Step 3: Filter Beach Hotels

Only hotels with:

hotel_type = 'Beach'

should be included.

Step 4: Select Required Columns

Return:

customer_name

hotel_name

city

Step 5: Sort the Result

Arrange alphabetically by:

customer_name

using:

ORDER BY customer_name

SQL Query

SELECT

c.customer_name,

h.hotel_name,

h.city

FROM customers c

JOIN reservations r

ON c.customer_id = r.customer_id

JOIN hotels h

ON r.hotel_id = h.hotel_id

WHERE r.reservation_status = 'Confirmed'

AND h.hotel_type = 'Beach'

ORDER BY c.customer_name;

Dry Run

reservations

reservation_id	hotel_id	status
1	1	Confirmed
2	2	Confirmed
3	3	Confirmed
4	4	Confirmed
5	1	Cancelled

hotels

hotel_id	hotel_type
1	Beach
2	City
3	Beach
4	Mountain

Processing:

- Reservation 1 → Included
- Reservation 2 → City Hotel → Excluded
- Reservation 3 → Included
- Reservation 4 → Mountain Hotel → Excluded
- Reservation 5 → Cancelled → Excluded

Sample Output

customer_name	hotel_name	city
Alice Sharma	Ocean View Resort	Goa
Charlie Roy	Palm Paradise Resort	Maldives

Concepts Tested

- INNER JOIN
- Multiple Table Joins
- WHERE Clause
- Multiple Conditions
- String Filtering
- ORDER BY

SELECT

c.customer_name,
h.hotel_name,
h.city

FROM customers c

JOIN reservations r

ON c.customer_id = r.customer_id

JOIN hotels h

ON r.hotel_id = h.hotel_id

WHERE r.reservation_status = 'Confirmed'

AND h.hotel_type = 'Beach'

ORDER BY c.customer_name;

QUESTION 2

ACTIVE SUBSCRIPTIONS FOR PREMIUM USERS

Problem Statement

StreamMax is a leading online streaming platform that offers various subscription plans to its users. The marketing team wants to identify premium users who currently have active subscriptions so they can target them with loyalty rewards and special promotional offers.

Write an SQL query to retrieve the details of premium users who have active subscriptions.

The report should display:

- user_name – Name of the user
- plan_name – Subscription plan
- expiry_date – Subscription expiry date

The result must be sorted alphabetically by user name.

Table Schema

Table: users

Column Name	Data Type	Description
user_id	INT	Unique user identifier
user_name	VARCHAR(100)	User name
email	VARCHAR(100)	User email
membership_type	VARCHAR(20)	Premium or Standard

Table: subscriptions

Column Name	Data Type	Description
subscription_id	INT	Unique subscription ID
user_id	INT	References users(user_id)
plan_name	VARCHAR(50)	Subscription plan
status	VARCHAR(20)	Active or Expired
expiry_date	DATE	Expiry date

Sample Input

Sample rows from users

user_id	user_name	membership_type
1	Alice	Premium
2	Bob	Standard
3	Charlie	Premium
4	David	Premium
5	Emma	Standard

Sample rows from subscriptions

subscription_id	user_id	plan_name	status	expiry_date
1	1	Gold	Active	2025-12-31
2	2	Silver	Active	2025-08-15
3	3	Platinum	Active	2026-01-01
4	4	Gold	Expired	2024-12-31
5	5	Silver	Active	2025-09-30

Expected Output Format

user_name plan_name expiry_date

Sample Output

user_name	plan_name	expiry_date
Alice	Gold	2025-12-31
Charlie	Platinum	2026-01-01

Explanation

User 1

Premium User ✓

Active Subscription ✓

Included

User 2

Standard User X

Excluded

User 3

Premium User ✓

Active Subscription ✓

Included

User 4

Premium User ✓

Expired Subscription X

Excluded

User 5

Standard User X

Excluded

Approach

Step 1: Join the Tables

We need data from:

users → subscriptions

Join Condition:

u.user_id = s.user_id

Step 2: Filter Premium Users

Only users with:

membership_type = 'Premium'

should be included.

Step 3: Filter Active Subscriptions

Only subscriptions with:
status = 'Active'
should be included.

Step 4: Select Required Columns

Return:
user_name
plan_name
expiry_date

Step 5: Sort the Result

Arrange alphabetically by:
user_name
using:
ORDER BY user_name

SQL Query

```
SELECT u.user_name, s.plan_name, s.expiry_date
FROM users u
JOIN subscriptions s ON u.user_id = s.user_id
WHERE u.membership_type = 'Premium' AND s.status = 'Active'
ORDER BY u.user_name;
```

Dry Run

subscriptions

user_id	plan_name	status
1	Gold	Active
2	Silver	Active
3	Platinum	Active
4	Gold	Expired

Processing:

User 1 → Included

User 2 → Standard → Excluded

User 3 → Included

User 4 → Expired → Excluded

SELECT

u.user_name, s.plan_name, s.expiry_date

FROM users u

JOIN subscriptions s

ON u.user_id = s.user_id

WHERE u.membership_type = 'Premium' AND s.status = 'Active'

ORDER BY u.user_name;

QUESTION 3

APPROVED LOAN APPLICATIONS ABOVE ₹500,000

Problem Statement

SafeBank is conducting an analysis of high-value loan approvals. The loan department wants to identify customers whose loan applications have been approved and whose loan amount exceeds ₹500,000.

Write an SQL query to retrieve the details of customers with approved high-value loans.

The report should display:

- customer_name – Name of the customer
- loan_type – Type of loan
- loan_amount – Approved loan amount

The result must be sorted alphabetically by customer name.

Table Schema

Table: customers

Column Name	Data Type	Description
customer_id	INT	Unique customer identifier
customer_name	VARCHAR(100)	Customer name
email	VARCHAR(100)	Customer email
city	VARCHAR(50)	Customer city

Table: loans

Column Name	Data Type	Description
loan_id	INT	Unique loan identifier
customer_id	INT	References customers(customer_id)
loan_type	VARCHAR(50)	Home, Auto, Personal, Education
loan_amount	DECIMAL(12,2)	Loan amount
loan_status	VARCHAR(20)	Approved or Rejected

Sample Input

Sample rows from customers

customer_id	customer_name	city
1	Alice Sharma	Mumbai
2	Bob Mehta	Delhi
3	Charlie Roy	Pune
4	David Khan	Chennai
5	Emma Patel	Bangalore

Sample rows from loans

loan_id	customer_id	loan_type	loan_amount	loan_status
1	1	Home	800000	Approved
2	2	Auto	450000	Approved
3	3	Personal	600000	Approved
4	4	Home	900000	Rejected
5	5	Education	700000	Approved

Expected Output Format

customer_name loan_type loan_amount

Sample Output

customer_name	loan_type	loan_amount
Alice Sharma	Home	800000
Charlie Roy	Personal	600000
Emma Patel	Education	700000

Explanation**Loan 1**

Approved ✓

Amount > 500000 ✓

Included

Loan 2

Approved ✓

Amount <= 500000 ✗

Excluded

Loan 3

Approved ✓

Amount > 500000 ✓

Included

Loan 4

Rejected ✗

Excluded

Loan 5

Approved ✓

Amount > 500000 ✓

Included

Approach**Step 1: Join the Tables**

We need data from:

customers → loans

Join Condition:

c.customer_id = l.customer_id

Step 2: Filter Approved Loans

Only loans with:

loan_status = 'Approved'

should be included.

Step 3: Filter High-Value Loans

Only loans where:

loan_amount > 500000

should be included.

Step 4: Select Required Columns

Return:

customer_name

loan_type

loan_amount

Step 5: Sort the Result

Arrange alphabetically by:

customer_name

using:

ORDER BY customer_name

SQL Query

SELECT

c.customer_name, l.loan_type, l.loan_amount

FROM customers c JOIN loans l

ON c.customer_id = l.customer_id

WHERE l.loan_status = 'Approved'

AND l.loan_amount > 500000

ORDER BY c.customer_name;

Dry Run

loans

loan_id	customer_id	amount	status
1	1	800000	Approved
2	2	450000	Approved
3	3	600000	Approved
4	4	900000	Rejected
5	5	700000	Approved

Processing:

Loan 1 → Included

Loan 2 → Amount condition fails

Loan 3 → Included

Loan 4 → Rejected

Loan 5 → Included

Sample Output

customer_name	loan_type	loan_amount
Alice Sharma	Home	800000
Charlie Roy	Personal	600000
Emma Patel	Education	700000

SELECT

c.customer_name,

l.loan_type,

l.loan_amount

FROM customers c JOIN loans l

ON c.customer_id = l.customer_id

WHERE l.loan_status = 'Approved' AND l.loan_amount > 500000

ORDER BY c.customer_name;

QUESTION 4

DELIVERED ORDERS FROM ELECTRONICS CATEGORY

Problem Statement

ShopKart is an online marketplace that sells products across multiple categories. The operations team wants to identify customers whose orders from the **Electronics** category have been successfully delivered.

Write an SQL query to retrieve the details of customers who received delivered electronics products.

The report should display:

- customer_name – Name of the customer
- product_name – Name of the product
- order_date – Date when the order was placed

The result must be sorted alphabetically by customer name.

Table Schema

Table: customers

Column Name	Data Type	Description
customer_id	INT	Unique customer identifier
customer_name	VARCHAR(100)	Customer name
city	VARCHAR(50)	Customer city

Table: products

Column Name	Data Type	Description
product_id	INT	Unique product identifier
product_name	VARCHAR(100)	Product name
category	VARCHAR(50)	Product category

Table: orders

Column Name	Data Type	Description
order_id	INT	Unique order identifier
customer_id	INT	References customers(customer_id)
product_id	INT	References products(product_id)
order_date	DATE	Date of order
status	VARCHAR(20)	Delivered / Cancelled / Pending

Sample Input

customers

customer_id customer_name

1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan

products

product_id	product_name	category
1	Laptop	Electronics
2	Mobile	Electronics
3	T-Shirt	Clothing
4	Tablet	Electronics

orders

order_id	customer_id	product_id	order_date	status
1	1	1	2025-06-01	Delivered
2	2	3	2025-06-02	Delivered
3	3	2	2025-06-03	Pending
4	4	4	2025-06-04	Delivered

Expected Output Format

customer_name
product_name
order_date

Sample Output

customer_name	product_name	order_date
Alice Sharma	Laptop	2025-06-01
David Khan	Tablet	2025-06-04

Explanation

Order 1

Delivered ✓
Electronics ✓
Included

Order 2

Delivered ✓
Electronics X
Excluded

Order 3

Pending X
Excluded

Order 4

Delivered ✓
Electronics ✓
Included

Approach

Step 1: Join Tables

customers → orders → products

Join Conditions:

c.customer_id = o.customer_id

o.product_id = p.product_id

Step 2: Filter Delivered Orders

status = 'Delivered'

Step 3: Filter Electronics Products

category = 'Electronics'

Step 4: Select Required Columns

customer_name

product_name

order_date

Step 5: Sort Output

ORDER BY customer_name

SQL Query

```
SELECT  c.customer_name,  p.product_name,  o.order_date
FROM customers c JOIN orders o
    ON c.customer_id = o.customer_id
JOIN products p    ON o.product_id = p.product_id
WHERE o.status = 'Delivered' AND p.category = 'Electronics'
ORDER BY c.customer_name;
```

```
SELECT  c.customer_name,  p.product_name,  o.order_date
FROM customers c
JOIN orders o    ON c.customer_id = o.customer_id
JOIN products p  ON o.product_id = p.product_id
WHERE o.status = 'Delivered' AND p.category = 'Electronics'
ORDER BY c.customer_name;
```

QUESTION 5

COMPLETED APPOINTMENTS WITH SPECIALIST DOCTORS

Problem Statement

CityCare Hospital wants to analyze patients who completed appointments with specialist doctors.

The hospital administration plans to send follow-up care recommendations to these patients.

Write an SQL query to retrieve patients who completed appointments with specialist doctors.

The report should display:

- patient_name – Name of the patient
- doctor_name – Name of the doctor
- specialization – Doctor specialization

The result must be sorted alphabetically by patient name.

Table Schema

Table: patients

Column Name	Data Type	Description
patient_id	INT	Unique patient identifier
patient_name	VARCHAR(100)	Patient name
city	VARCHAR(50)	Patient city

Table: doctors

Column Name	Data Type	Description
doctor_id	INT	Unique doctor identifier
doctor_name	VARCHAR(100)	Doctor name
specialization	VARCHAR(50)	Medical specialization

Table: appointments

Column Name	Data Type	Description
appointment_id	INT	Unique appointment identifier
patient_id	INT	References patients(patient_id)
doctor_id	INT	References doctors(doctor_id)
appointment_date	DATE	Appointment date
status	VARCHAR(20)	Completed / Cancelled

Sample Input

patients

patient_id	patient_name
1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan

doctors

doctor_id	doctor_name	specialization
1	Dr. Verma	Cardiologist
2	Dr. Singh	General
3	Dr. Patel	Neurologist

appointments

appointment_id	patient_id	doctor_id	status
1	1	1	Completed
2	2	2	Completed
3	3	3	Completed
4	4	1	Cancelled

Expected Output Format

patient_name doctor_name specialization

Sample Output

patient_name	doctor_name	specialization
Alice Sharma	Dr. Verma	Cardiologist
Charlie Roy	Dr. Patel	Neurologist

Explanation

Appointment 1

Completed ✓

Specialist ✓

Included

Appointment 2

Completed ✓

General Doctor X

Excluded

Appointment 3

Completed ✓

Specialist ✓

Included

Appointment 4

Cancelled X

Excluded

Approach

Step 1: Join Tables

patients → appointments → doctors

Join Conditions:

p.patient_id = a.patient_id

a.doctor_id = d.doctor_id

Step 2: Filter Completed Appointments

status = 'Completed'

Step 3: Exclude General Doctors

specialization <> 'General'

Step 4: Select Required Columns

patient_name

doctor_name

specialization

Step 5: Sort Results

ORDER BY patient_name

SQL Query

```
SELECT p.patient_name, d.doctor_name, d.specialization
FROM patients p JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE a.status = 'Completed' AND d.specialization <> 'General'
ORDER BY p.patient_name;
```

Final Query

```
SELECT p.patient_name, d.doctor_name, d.specialization
FROM patients p JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE a.status = 'Completed' AND d.specialization <> 'General'
ORDER BY p.patient_name;
```


QUESTION 6

REGISTERED PARTICIPANTS IN PAID EVENTS

Problem Statement

EventHub is an online event management platform that organizes conferences, workshops, and training programs.

The marketing team wants to identify participants who have successfully registered for **paid events** so they can send event reminders and promotional offers.

Write an SQL query to retrieve the details of participants who are registered for paid events.

The report should display:

- participant_name – Name of the participant
- event_name – Name of the event
- event_fee – Registration fee of the event

The result must be sorted alphabetically by participant name.

Table Schema

Table: participants

Column Name	Data Type	Description
participant_id	INT	Unique participant identifier
participant_name	VARCHAR(100)	Participant name
email	VARCHAR(100)	Participant email
city	VARCHAR(50)	Participant city

Table: events

Column Name	Data Type	Description
event_id	INT	Unique event identifier
event_name	VARCHAR(100)	Event name
event_fee	DECIMAL(10,2)	Registration fee
event_type	VARCHAR(50)	Workshop, Conference, Seminar

Table: registrations

Column Name	Data Type	Description
registration_id	INT	Unique registration identifier
participant_id	INT	References participants(participant_id)
event_id	INT	References events(event_id)
registration_status	VARCHAR(20)	Registered / Cancelled
registration_date	DATE	Registration date

Sample Input

Sample rows from participants

participant_id	participant_name
1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan
5	Emma Patel

Sample rows from events

event_id	event_name	event_fee	event_type
1	Data Science Workshop	5000	Workshop
2	SQL Masterclass	0	Seminar
3	AI Conference	8000	Conference
4	Cloud Summit	10000	Conference

Sample rows from registrations

registration_id	participant_id	event_id	registration_status
1	1	1	Registered
2	2	2	Registered
3	3	3	Registered
4	4	4	Cancelled
5	5	1	Registered

Expected Output Format

participant_name event_name event_fee

Sample Output

participant_name	event_name	event_fee
Alice Sharma	Data Science Workshop	5000
Charlie Roy	AI Conference	8000
Emma Patel	Data Science Workshop	5000

Explanation**Registration 1**

Registered ✓

Paid Event ✓

Included

Registration 2

Registered ✓

Paid Event X

Excluded

(SQL Masterclass fee = 0)

Registration 3

Registered ✓

Paid Event ✓

Included

Registration 4

Cancelled X

Excluded

Registration 5

Registered ✓

Paid Event ✓

Included

Approach

Step 1: Join the Tables

We need data from all three tables:

participants → registrations → events

Join Conditions:

p.participant_id = r.participant_id

r.event_id = e.event_id

Step 2: Filter Registered Participants

Only registrations with:

registration_status = 'Registered'

should be included.

Step 3: Filter Paid Events

Only events with:

event_fee > 0

should be included.

Step 4: Select Required Columns

Return:

participant_name

event_name

event_fee

Step 5: Sort the Result

Arrange alphabetically by:

participant_name

using:

ORDER BY participant_name

SQL Query

```
SELECT p.participant_name, e.event_name, e.event_fee
```

```
FROM participants p JOIN registrations r
```

```
ON p.participant_id = r.participant_id
```

```
JOIN events e
```

```
ON r.event_id = e.event_id
```

```
WHERE r.registration_status = 'Registered' AND e.event_fee > 0
```

```
ORDER BY p.participant_name;
```

Dry Run

registrations

participant_id	event_id	status
1	1	Registered
2	2	Registered
3	3	Registered
4	4	Cancelled
5	1	Registered

events

event_id	fee
1	5000
2	0
3	8000
4	10000

Processing:

- Registration 1 → Included
- Registration 2 → Free Event → Excluded
- Registration 3 → Included
- Registration 4 → Cancelled → Excluded
- Registration 5 → Included

Concepts Tested

- INNER JOIN
- Multiple Table Join
- WHERE
- Numeric Filtering
- ORDER BY

SELECT

```
p.participant_name, e.event_name, e.event_fee
FROM participants p JOIN registrations r
  ON p.participant_id = r.participant_id
JOIN events e    ON r.event_id = e.event_id
WHERE r.registration_status = 'Registered' AND e.event_fee > 0
ORDER BY p.participant_name;
```

QUESTION 7

INTERNATIONAL SHIPMENTS DELIVERED SUCCESSFULLY

Problem Statement

GlobalExpress is an international logistics company that handles shipments across multiple countries. The operations team wants to identify customers whose **international shipments** have been successfully delivered so they can gather customer feedback and improve service quality. Write an SQL query to retrieve details of customers whose international shipments were delivered successfully.

The report should display:

- customer_name – Name of the customer
- tracking_number – Shipment tracking number
- destination_country – Destination country of the shipment

The result must be sorted alphabetically by customer name.

Table Schema

Table: customers

Column Name	Data Type	Description
customer_id	INT	Unique customer identifier
customer_name	VARCHAR(100)	Customer name
email	VARCHAR(100)	Customer email
country	VARCHAR(50)	Country of residence

Table: shipments

Column Name	Data Type	Description
shipment_id	INT	Unique shipment identifier
customer_id	INT	References customers(customer_id)
tracking_number	VARCHAR(50)	Shipment tracking number
shipment_type	VARCHAR(20)	Domestic / International
destination_country	VARCHAR(50)	Destination country
status	VARCHAR(20)	Delivered / In Transit / Cancelled

Sample Input

Sample rows from customers

customer_id	customer_name
1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan
5	Emma Patel

Sample rows from shipments

shipment_id	customer_id	tracking_number	shipment_type	destination_country	status
1	1	TRK1001	International	USA	Delivered
2	2	TRK1002	Domestic	India	Delivered
3	3	TRK1003	International	Germany	Delivered
4	4	TRK1004	International	Singapore	In Transit
5	5	TRK1005	International	Canada	Delivered

Expected Output Format

customer_name tracking_number destination_country

Sample Output

customer_name	tracking_number	destination_country
Alice Sharma	TRK1001	USA
Charlie Roy	TRK1003	Germany
Emma Patel	TRK1005	Canada

Explanation

Shipment 1

International ✓

Delivered ✓

Included

Shipment 2

Domestic X

Excluded

Shipment 3

International ✓

Delivered ✓

Included

Shipment 4

In Transit X

Excluded

Shipment 5

International ✓

Delivered ✓

Included

Approach

Step 1: Join the Tables

We need data from:

customers → shipments

Join Condition:

c.customer_id = s.customer_id

Step 2: Filter International Shipments

Only shipments with:
shipment_type = 'International'
should be included.

Step 3: Filter Delivered Shipments

Only shipments with:
status = 'Delivered'
should be included.

Step 4: Select Required Columns

Return:
customer_name tracking_number destination_country

Step 5: Sort the Result

Arrange alphabetically by:
customer_name
using:
ORDER BY customer_name

SQL Query

```
SELECT c.customer_name, s.tracking_number, s.destination_country
FROM customers c JOIN shipments s ON c.customer_id = s.customer_id
WHERE s.shipment_type = 'International' AND s.status = 'Delivered'
ORDER BY c.customer_name;
```

Dry Run

shipments

shipment_id	type	status
1	International	Delivered
2	Domestic	Delivered
3	International	Delivered
4	International	In Transit
5	International	Delivered

Processing:

- Shipment 1 → Included
- Shipment 2 → Domestic → Excluded
- Shipment 3 → Included
- Shipment 4 → In Transit → Excluded
- Shipment 5 → Included

Sample Output

customer_name	tracking_number	destination_country
Alice Sharma	TRK1001	USA
Charlie Roy	TRK1003	Germany
Emma Patel	TRK1005	Canada

```
SELECT c.customer_name, s.tracking_number, s.destination_country
FROM customers c JOIN shipments s ON c.customer_id = s.customer_id
WHERE s.shipment_type = 'International' AND s.status = 'Delivered'
ORDER BY c.customer_name;
```

QUESTION 8

EMPLOYEES ASSIGNED TO ACTIVE PROJECTS

Problem Statement

TechNova Solutions is managing multiple projects across various departments. The Human Resources team wants to identify employees who are currently assigned to **active projects** so they can evaluate resource allocation and project performance.

Write an SQL query to retrieve the details of employees working on active projects.

The report should display:

- employee_name – Name of the employee
- project_name – Name of the project
- department_name – Department to which the employee belongs

The result must be sorted alphabetically by employee name.

Table Schema

Table: departments

Column Name	Data Type	Description
department_id	INT	Unique department identifier
department_name	VARCHAR(100)	Department name

Table: projects

Column Name	Data Type	Description
project_id	INT	Unique project identifier
project_name	VARCHAR(100)	Project name
project_status	VARCHAR(20)	Active / Completed / On Hold

Table: employees

Column Name	Data Type	Description
employee_id	INT	Unique employee identifier
employee_name	VARCHAR(100)	Employee name
department_id	INT	References departments(department_id)
project_id	INT	References projects(project_id)

Sample Input

Sample rows from departments

department_id	department_name
1	IT
2	HR
3	Finance

Sample rows from projects

project_id	project_name	project_status
1	ERP Upgrade	Active
2	Payroll Automation	Completed
3	Cloud Migration	Active
4	AI Analytics	On Hold

Sample rows from employees

employee_id	employee_name	department_id	project_id
1	Alice Sharma	1	1
2	Bob Mehta	2	2
3	Charlie Roy	1	3
4	David Khan	3	4
5	Emma Patel	1	1

Expected Output Format

employee_name project_name department_name

Sample Output

employee_name	project_name	department_name
Alice Sharma	ERP Upgrade	IT
Charlie Roy	Cloud Migration	IT
Emma Patel	ERP Upgrade	IT

Explanation

Employee 1

Assigned to Active Project ✓

Included

Employee 2

Project Completed X

Excluded

Employee 3

Assigned to Active Project ✓

Included

Employee 4

Project On Hold X

Excluded

Employee 5

Assigned to Active Project ✓

Included

Approach

Step 1: Join the Tables

We need data from:

employees → departments

employees → projects

Join Conditions:

e.department_id = d.department_id

e.project_id = p.project_id

Step 2: Filter Active Projects

Only projects with: project_status = 'Active'

should be included.

Step 3: Select Required Columns

Return:

employee_name project_name department_name

Step 4: Sort the Result

Arrange alphabetically by:

employee_name

using:

ORDER BY employee_name

SQL Query

```
SELECT e.employee_name, p.project_name, d.department_name
FROM employees e JOIN departments d
    ON e.department_id = d.department_id
JOIN projects p    ON e.project_id = p.project_id
WHERE p.project_status = 'Active'
ORDER BY e.employee_name;
```

Dry Run

projects

project_id	project_name	project_status
1	ERP Upgrade	Active
2	Payroll Automation	Completed
3	Cloud Migration	Active
4	AI Analytics	On Hold

Processing:

- Employee 1 → Active Project → Included
 - Employee 2 → Completed Project → Excluded
 - Employee 3 → Active Project → Included
 - Employee 4 → On Hold Project → Excluded
 - Employee 5 → Active Project → Included
-

Sample Output

employee_name	project_name	department_name
Alice Sharma	ERP Upgrade	IT
Charlie Roy	Cloud Migration	IT
Emma Patel	ERP Upgrade	IT

```
SELECT
    e.employee_name,
    p.project_name,
    d.department_name
FROM employees e
JOIN departments d
    ON e.department_id = d.department_id
JOIN projects p
    ON e.project_id = p.project_id
WHERE p.project_status = 'Active'
ORDER BY e.employee_name;
```

QUESTION 9

CUSTOMERS WITH SUCCESSFUL PAYMENTS ABOVE ₹10,000

Problem Statement

PayEase is a digital payment platform that processes online transactions for customers across multiple merchants.

The finance team wants to identify customers who have made **successful payments greater than ₹10,000** so they can analyze high-value transactions and offer premium rewards.

Write an SQL query to retrieve details of customers whose successful payments exceed ₹10,000.

The report should display:

- customer_name – Name of the customer
- payment_amount – Amount paid
- payment_date – Date of payment

The result must be sorted alphabetically by customer name.

Table Schema

Table: customers

Column Name	Data Type	Description
customer_id	INT	Unique customer identifier
customer_name	VARCHAR(100)	Customer name
email	VARCHAR(100)	Customer email
city	VARCHAR(50)	Customer city

Table: payments

Column Name	Data Type	Description
payment_id	INT	Unique payment identifier
customer_id	INT	References customers(customer_id)
payment_amount	DECIMAL(12,2)	Payment amount
payment_date	DATE	Date of payment
payment_status	VARCHAR(20)	Success / Failed / Pending

Sample Input

Sample rows from customers

customer_id	customer_name
1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan
5	Emma Patel

Sample rows from payments

payment_id	customer_id	payment_amount	payment_date	payment_status
1	1	15000	2025-06-01	Success
2	2	8500	2025-06-02	Success
3	3	25000	2025-06-03	Success
4	4	18000	2025-06-04	Failed
5	5	12000	2025-06-05	Success

Expected Output Format

customer_name payment_amount payment_date

Sample Output

customer_name	payment_amount	payment_date
Alice Sharma	15000	2025-06-01
Charlie Roy	25000	2025-06-03
Emma Patel	12000	2025-06-05

Explanation

Payment 1

Success ✓

Amount > 10000 ✓

Included

Payment 2

Success ✓

Amount <= 10000 ✗

Excluded

Payment 3

Success ✓

Amount > 10000 ✓

Included

Payment 4

Failed ✗

Excluded

Payment 5

Success ✓

Amount > 10000 ✓

Included

Approach

Step 1: Join the Tables

We need data from:

customers → payments

Join Condition:

c.customer_id = p.customer_id

Step 2: Filter Successful Payments

Only payments with:
payment_status = 'Success'
should be included.

Step 3: Filter High-Value Payments

Only payments where:
payment_amount > 10000
should be included.

Step 4: Select Required Columns

Return:
customer_name payment_amount payment_date

Step 5: Sort the Result

Arrange alphabetically by:
customer_name
using:
ORDER BY customer_name

SQL Query

```
SELECT c.customer_name, p.payment_amount, p.payment_date
FROM customers c JOIN payments p ON c.customer_id = p.customer_id
WHERE p.payment_status = 'Success' AND p.payment_amount > 10000
ORDER BY c.customer_name;
```

Dry Run

payments

payment_id	amount	status
1	15000	Success
2	8500	Success
3	25000	Success
4	18000	Failed
5	12000	Success

Processing:

- Payment 1 → Included
- Payment 2 → Amount condition fails
- Payment 3 → Included
- Payment 4 → Failed
- Payment 5 → Included

Sample Output

customer_name	payment_amount	payment_date
Alice Sharma	15000	2025-06-01
Charlie Roy	25000	2025-06-03
Emma Patel	12000	2025-06-05

```
SELECT c.customer_name, p.payment_amount, p.payment_date
FROM customers c JOIN payments p ON c.customer_id = p.customer_id
WHERE p.payment_status = 'Success' AND p.payment_amount > 10000
ORDER BY c.customer_name;
```

QUESTION 10

PASSENGERS ON BUSINESS CLASS INTERNATIONAL FLIGHTS

Problem Statement

SkyLink Airlines operates both domestic and international flights across multiple destinations.

The airline's premium services team wants to identify passengers who have booked **Business Class seats on international flights**. These passengers will be targeted for exclusive lounge access, loyalty rewards, and special travel offers.

Write an SQL query to retrieve details of passengers who have booked Business Class seats on international flights.

The report should display:

- passenger_name – Name of the passenger
- flight_number – Airline-assigned flight number
- destination – Arrival city of the flight

The result must be sorted alphabetically by passenger name.

Table Schema

Table: passengers

Column Name	Data Type	Description
passenger_id	INT	Unique passenger identifier
passenger_name	VARCHAR(100)	Passenger name
email	VARCHAR(100)	Passenger email
country	VARCHAR(50)	Country of residence

Table: flights

Column Name	Data Type	Description
flight_id	INT	Unique flight identifier
flight_number	VARCHAR(20)	Airline-assigned flight number
origin	VARCHAR(50)	Departure city
destination	VARCHAR(50)	Arrival city
is_international	VARCHAR(1)	'Y' for International, 'N' for Domestic
base_price	DECIMAL(10,2)	Base ticket price

Table: bookings

Column Name	Data Type	Description
booking_id	INT	Unique booking identifier
passenger_id	INT	References passengers(passenger_id)
flight_id	INT	References flights(flight_id)
seat_class	VARCHAR(20)	Economy, Business, First
booking_status	VARCHAR(20)	Confirmed, Cancelled
booking_date	DATE	Date of booking

Sample Input

Sample rows from passengers

passenger_id	passenger_name
1	Alice Sharma
2	Bob Mehta
3	Charlie Roy
4	David Khan
5	Emma Patel

Sample rows from flights

flight_id	flight_number	destination	is_international
1	SK101	Mumbai	N
2	SK202	London	Y
3	SK303	Singapore	Y
4	SK404	Dubai	Y
5	SK505	Delhi	N

Sample rows from bookings

booking_id	passenger_id	flight_id	seat_class	booking_status
1	1	2	Business	Confirmed
2	2	1	Business	Confirmed
3	3	3	Economy	Confirmed
4	4	4	Business	Confirmed
5	5	2	Business	Cancelled

Expected Output Format

passenger_name flight_number destination

Sample Output

passenger_name	flight_number	destination
Alice Sharma	SK202	London
David Khan	SK404	Dubai

Explanation

Booking 1

Business Class ✓

International Flight ✓

Confirmed Booking ✓

Included

Booking 2

Business Class ✓

International Flight X

Excluded

(Domestic flight)

Booking 3

Economy Class X

Excluded

Booking 4

Business Class ✓

International Flight ✓

Confirmed Booking ✓

Included

Booking 5

Cancelled Booking X

Excluded

Approach**Step 1: Join the Tables**

We need data from:

passengers → bookings → flights

Join Conditions:

p.passenger_id = b.passenger_id

b.flight_id = f.flight_id

Step 2: Filter Business Class Bookings

Only bookings with:

seat_class = 'Business'

should be included.

Step 3: Filter International Flights

Only flights with:

is_international = 'Y'

should be included.

Step 4: Filter Confirmed Bookings

Only bookings with:

booking_status = 'Confirmed'

should be included.

Step 5: Select Required Columns

Return:

passenger_name

flight_number

destination

Step 6: Sort the Result

Arrange alphabetically by:

passenger_name

using:

ORDER BY passenger_name

SQL Query

```
SELECT  p.passenger_name,  f.flight_number,  f.destination
FROM    passengers p JOIN bookings b
        ON p.passenger_id = b.passenger_id
JOIN    flights f  ON b.flight_id = f.flight_id
WHERE   b.seat_class = 'Business' AND b.booking_status = 'Confirmed' AND f.is_international = 'Y'
ORDER BY p.passenger_name;
```

Dry Run

bookings

booking_id	seat_class	status
1	Business	Confirmed
2	Business	Confirmed
3	Economy	Confirmed
4	Business	Confirmed
5	Business	Cancelled

flights

flight_id	international
1	N
2	Y
3	Y
4	Y

Processing:

- Booking 1 → Included
 - Booking 2 → Domestic Flight → Excluded
 - Booking 3 → Economy Class → Excluded
 - Booking 4 → Included
 - Booking 5 → Cancelled → Excluded
-

Sample Output

passenger_name	flight_number	destination
Alice Sharma	SK202	London
David Khan	SK404	Dubai

```
SELECT
  p.passenger_name,  f.flight_number,  f.destination
FROM    passengers p JOIN bookings b  ON p.passenger_id = b.passenger_id
JOIN    flights f  ON b.flight_id = f.flight_id
WHERE   b.seat_class = 'Business' AND b.booking_status = 'Confirmed' AND f.is_international = 'Y'
ORDER BY p.passenger_name;
```
